1. Создайте скрипт на языке bash с именем cuckoo.sh, делающий следующее:

- Создает именованный канал с именем cuckoo.pid в каталоге /tmp/run/
- Записывает в файл cuckoo.log сообщение в формате: ДАТА ВРЕМЯ Startup!
- В процессе ожидания читает сообщения из именованного канала, на запрос «NAME[PID]: how much time do I have?" возвращает число N в диапазоне от 2 до 10, где NAME – имя обратившегося процесса и PID – его идентификатор, записывает обращение и ответ в файл cuckoo.log в формате: ДАТА ВРЕМЯ NAME[PID] N
- При получении сигнала (на ваш выбор) скрипт записывает в файл cuckoo.log сообщение в формате: ДАТА ВРЕМЯ Shutdown! и завершает свою работу.

Проверка наличия папки /tmp/run:

```
user1@eltex-practice2-pg1-v22:/tmp$ ls --file-type
archive_bz2_list.txt
dir_etc_backup_list.txt
snap-private-tmp/
systemd-private-a293a8dc047d4a03a46ccc3356e5fd21-fwupd.service-ebY2nl/
systemd-private-a293a8dc047d4a03a46ccc3356e5fd21-ModemManager.service-7Kbojm/
systemd-private-a293a8dc047d4a03a46ccc3356e5fd21-polkit.service-OlUBK7/
systemd-private-a293a8dc047d4a03a46ccc3356e5fd21-systemd-logind.service-oyAZ71/
systemd-private-a293a8dc047d4a03a46ccc3356e5fd21-systemd-resolved.service-XEfB0J/
systemd-private-a293a8dc047d4a03a46ccc3356e5fd21-systemd-timesyncd.service-zbXinA/
systemd-private-a293a8dc047d4a03a46ccc3356e5fd21-upower.service-fcPi4e/
user1@eltex-practice2-pg1-v22:/tmp$
```

Создание файла cuckoo.sh:

```
user1@eltex-practice2-pg1-v22: ~
#!/bin/bash

FIFO="/tmp/run/cuckoo.pid"
LOGFILE="cuckoo.log"

mkdir -p /tmp/run

# Если канал уже существует, удалим его
[ -p "$FIFO" ] && rm -f "$FIFO"
mkfifo "$FIFO"

log_msg(){
        echo "$(date '+%F %T') $1" >> "$LOGFILE"
}

shutdown_handler(){
        log_msg "Shutdown!"
        rm -f "$FIFO"
        exit 0
}

# Дополнительно можно завершать, через SIGTERM SIGINT
trap shutdown_handler SIGTERM SIGINT

log_msg "Startup!"

while true; do
    if read -r line < "$FIFO"; then

        if [[ $line == *": how much time do I have?" ]]; then
            prefix="${line%: how much time do I have?}"
            NAME="${prefix%%\[*}"
            PID="${prefix#*\[}"
            PID="${PID%\]}"

            if [[ -n $NAME && $PID =~ ^[0-9]+$ ]]; then
                N=$((RANDOM % 9 + 2))
                log_msg "${NAME}[${PID}] ${N}"
            fi
        fi
    fi
done
~
"cuckoo.sh" 42L, 939B written                          32,33         All
```

2. Запустите скрипт cuckoo.sh при помощи nohup или screen

```
user1@eltex-practice2-pg1-v22: ~
~
~
~
user1@eltex-practice2-pg1-v22:~$ chmod +x cuckoo.sh
user1@eltex-practice2-pg1-v22:~$ echo $STY
110646.cuckoo_see
user1@eltex-practice2-pg1-v22:~$ ./cuckoo.sh
```

Проверка:

```
user1@eltex-practice2-pg1-v22: ~
[detached from 111324.cuckoo_see]
user1@eltex-practice2-pg1-v22:~$ cat cuckoo.log
2026-04-01 08:11:28 Startup!
user1@eltex-practice2-pg1-v22:~$  echo "bash[$$]: how much time do I have?" > /tmp/run/cuckoo.pid
user1@eltex-practice2-pg1-v22:~$ cat cuckoo.log
2026-04-01 08:11:28 Startup!
2026-04-01 08:11:45 bash[110115] 5
user1@eltex-practice2-pg1-v22:~$
```

3. Создайте скрипт на языке bash с именем template_task.sh, делающий следующее:

- При запуске проверят, что имя скрипта не совпадает с template_task.sh, если совпадает - выходит с уведомлением «я бригадир, сам не работаю»
- При запуске дописывает в файл report_имя_скрипта_без_полного_пути.log в рабочем каталоге информацию: ДАТА ВРЕМЯ [PID] Скрипт запущен
- Запрашивает через именованный канал скрипта cuckoo время сообщением в указанном ранее формате и ждет такое количество секунд.
- Дописывает в файл report_имя_скрипта_без_полного_пути.log сообщение: ДАТА ВРЕМЯ [PID] Скрипт завершился, работал N секунд.

user1@eltex-practice2-pg1-v22: ~

```
#!/bin/bash

SCRIPT_NAME="$(basename "$0")"
BASE_NAME="${SCRIPT_NAME%.sh}"
REPORT_FILE="report_${BASE_NAME}.log"
FIFO="/tmp/run/cuckoo.pid"

# Проверка имени скрипта, на соответствие
if [[ "$SCRIPT_NAME" == "template_task.sh" ]]; then
        echo "я бригадир, сам не работаю"
        exit 1
fi

START_TIME=$(date +%s)

log_msg() {
        echo "$(date '+%F %T') [$$] $1" >> "$REPORT_FILE"
}

log_msg "Скрипт запущен"

# Отправляем запрос в cuckoo
echo "${BASE_NAME}[$$]: how much time do I have?" > "$FIFO"

# Ждем ответ от cuckoo из его лога
while true; do
        LAST_LINE=$(grep "${BASE_NAME}\[$$\]" cuckoo.log | tail -n 1)
        if [[ -n "$LAST_LINE" ]]; then
                N=$(echo "$LAST_LINE" | awk '{print $NF}')
                if [[ "$N" =~ ^[0-9]+$ ]]; then
                    break
                fi
        fi
        sleep 1
done

sleep "$N"
END_TIME=$(date +%s)
WORK_TIME=$((END_TIME - START_TIME))

log_msg "Скрипт завершился, работал ${WORK_TIME} секунд."

~
"template_task.sh" [New] 42L, 1013B written                    23,27         All
```

4. Создайте скрипт на языке bash с именем observer.sh, читающий файл конфигурации со списком скриптов observer.conf, проверяющим их наличие в списке работающих процессов поиском в /proc и запускающих их в отключенном от терминала режиме (nohup) в случае отсутствия в нем. Информация о перезапуске дописывайте в файл observer.log

```
user1@eltex-practice2-pg1-v22: ~
#!/bin/bash

SCRIPT_DIR="$(cd "$(dirname "$0")" && pwd)"
CONFIG_FILE="$SCRIPT_DIR/observer.conf"
LOG_FILE="$SCRIPT_DIR/observer.log"

log_msg() {
    echo "$(date '+%F %T') $1" >> "$LOG_FILE"
}

if [[ ! -f "$CONFIG_FILE" ]]; then
    echo "Файл $CONFIG_FILE не найден"
    exit 1
fi

while read -r script_name; do
    [[ -z "$script_name" || "$script_name" =~ ^# ]] && continue

    if ! pgrep -f "$script_name" > /dev/null; then
        nohup "$SCRIPT_DIR/$script_name" > /dev/null 2>&1 &
        log_msg "Перезапуск $script_name"
    fi
done < "$CONFIG_FILE"

~
~
"observ.sh" [New] 24L, 588B written                    23,21         All
```

5. Настройте запуск observer.sh посредством cron по расписанию – 1 раз в минуту

```
user1@eltex-practice2-pg1-v22: ~
user1@eltex-practice2-pg1-v22:~$ crontab -e
No modification made
user1@eltex-practice2-pg1-v22:~$ crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
* * * * * /home/user1/observer.sh
user1@eltex-practice2-pg1-v22:~$ chmod +x /home/user1/observ.sh
```

6. Создайте несколько символьных ссылок на файл template_task.sh с различными именами (рабочие задачи), добавьте в файл конфигурации observer.conf соответствующие записи об этих задачах, включая исходный файл template_task.sh

```
user1@eltex-practice2-pg1-v22: ~
user1@eltex-practice2-pg1-v22:~$ ln -s template_task.sh task1.sh
user1@eltex-practice2-pg1-v22:~$ ln -s template_task.sh task2.sh
user1@eltex-practice2-pg1-v22:~$ ln -s template_task.sh task3.sh
user1@eltex-practice2-pg1-v22:~$ cat > observ.conf <<'EOF'
> tepmplate_task.sh
> task1.sh
> task2.sh
> task3.sh
> EOF
user1@eltex-practice2-pg1-v22:~$ ls -l
total 20
-rw-rw-r-- 1 user1 user1   29 Apr  1 09:45 cuckoo.log
-rwxrwxr-x 1 user1 user1  939 Apr  1 08:08 cuckoo.sh
-rw-rw-r-- 1 user1 user1   45 Apr  1 09:48 observ.conf
-rwxrwxr-x 1 user1 user1  588 Apr  1 09:07 observ.sh
lrwxrwxrwx 1 user1 user1   16 Apr  1 09:38 task1.sh -> template_task.sh
lrwxrwxrwx 1 user1 user1   16 Apr  1 09:38 task2.sh -> template_task.sh
lrwxrwxrwx 1 user1 user1   16 Apr  1 09:39 task3.sh -> template_task.sh
-rwxrwxr-x 1 user1 user1 1013 Apr  1 08:31 template_task.sh
user1@eltex-practice2-pg1-v22:~$
```

7. Соберите статистику работы в виде набора файлов report_*.log, observer.log, cuckoo.log приложите их вместе с исходными текстами скриптов в качестве отчета в виде сжатого архива tar. Не забудьте остановить процесс, удалив задачу в cron!

```
user1@eltex-practice2-pg1-v22: ~
user1@eltex-practice2-pg1-v22:~$ crontab -e
crontab: installing new crontab
user1@eltex-practice2-pg1-v22:~$ crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
```

```
user1@eltex-practice2-pg1-v22: ~
user1@eltex-practice2-pg1-v22:~$ pkill -f cuckoo.sh
user1@eltex-practice2-pg1-v22:~$ ps -ef | grep '\.sh'
user1     112415  110102  0 10:13 pts/2    00:00:00 grep --color=auto \.sh
user1@eltex-practice2-pg1-v22:~$ tar -cvf report_archive.tar cuckoo.sh template_task.sh observer.sh observer.conf cuckoo.log observer.log report_*.log
cuckoo.sh
template_task.sh
observer.sh
observer.conf
cuckoo.log
observer.log
report_task1.log
report_task2.log
report_task3.log
user1@eltex-practice2-pg1-v22:~$
```